

Web Authentication Flow with JavaScript

Topics:

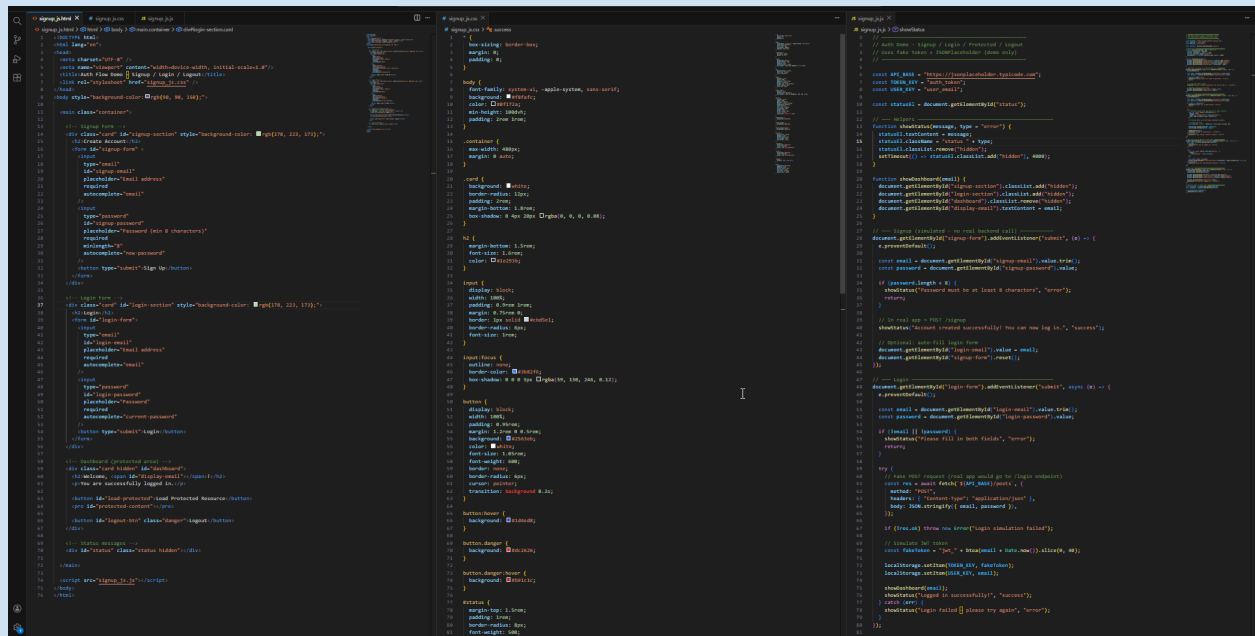
1. Login, Logout & Signup using JS (Fetch + Storage)
2. Introduction to APIs
3. Authorization Management (Tokens, Headers, Interceptors)
4. Developer Tools + Session Storage in Practice
5. Network Tab – Useful for Debugging & Hacking

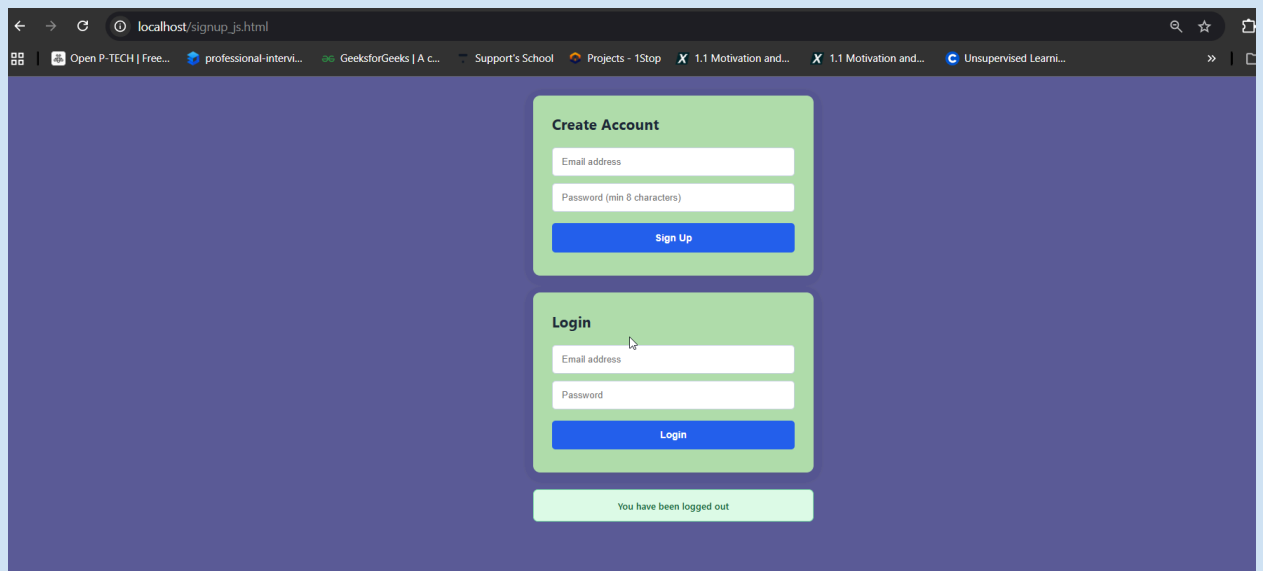
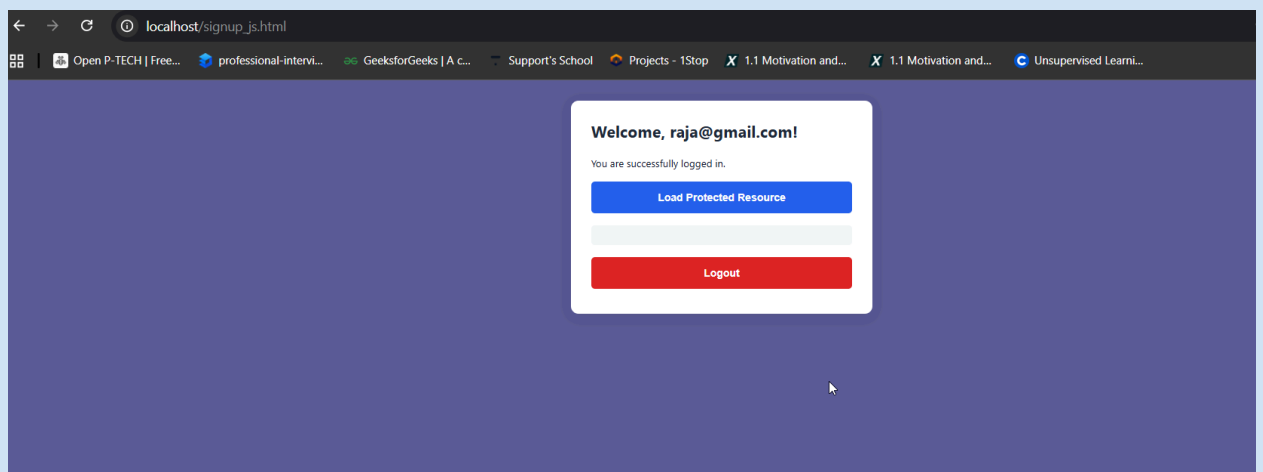
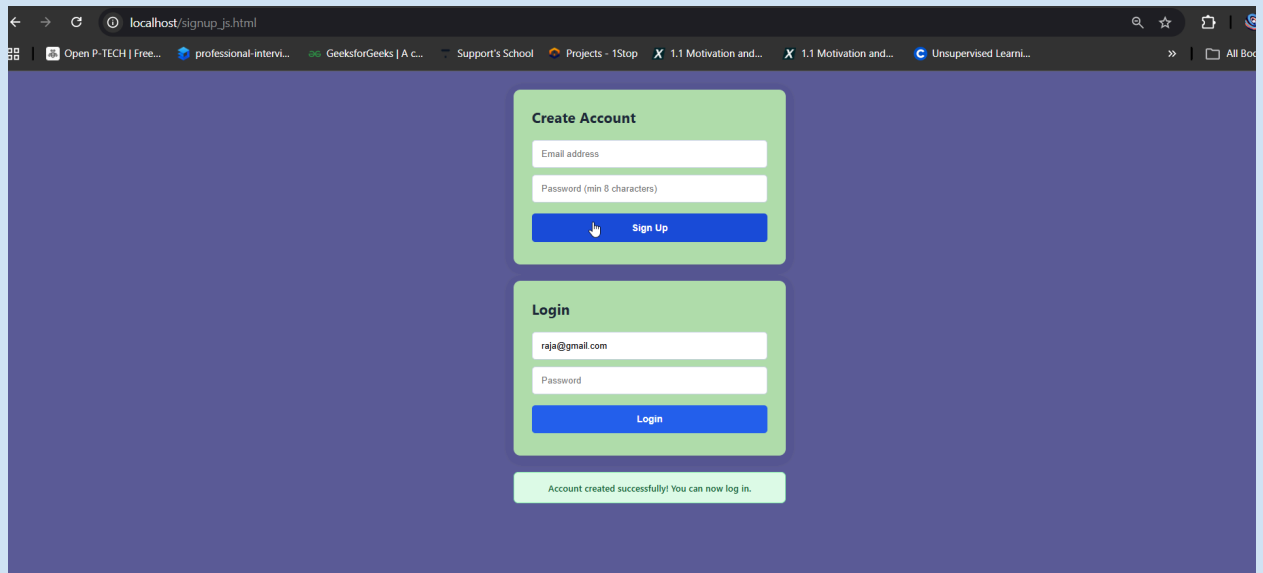
Login, Logout & Signup Functionalities using JavaScript

Core Flow

Signup → Login → Receive JWT → Store token → Use token in future requests → Logout (clear token)

Example (Frontend only – uses JSONPlaceholder API)





1. Introduction to APIs

API (Application Programming Interface) = contract between client and server for data exchange.

Types:

Type	Example	When used
REST	<code>/users, /posts/1</code>	Most common (JSON over HTTP)
GraphQL	single endpoint, query what you need	Complex data fetching
WebSocket	real-time chat, live updates	Bidirectional
gRPC	binary, high-performance	Microservices

REST basics

- Uses HTTP methods: GET, POST, PUT/PATCH, DELETE
- Returns JSON (usually)
- Stateless → each request must include auth info (usually token in header)

3. Authorization Management

Common patterns

1. **JWT (JSON Web Token)** – most popular 2025
 - Header.Payload.Signature
 - Stored client-side (localStorage or HttpOnly cookie)
 - Sent in **Authorization: Bearer <token>**
2. **Session ID** (traditional)
 - Server stores session
 - Client sends cookie
3. **OAuth2 / OpenID Connect**
 - For third-party login (Google, GitHub)

Best practices

- Prefer **HttpOnly + Secure + SameSite cookies** for tokens (prevents XSS theft)
- If using localStorage → be ready to handle XSS risk
- Always send token in Authorization header (not URL or body)
- Refresh tokens for long sessions
- Logout = invalidate on server + clear client storage

4. Developer Tools & Session Storage in Practice

DevTools (Chrome/Edge/Firefox)

- Elements → inspect & modify DOM (great for client-side bypass testing)
- Console → run JS, test payloads
- Sources → set breakpoints, debug
- **Application tab** →
 - Storage → localStorage, sessionStorage, cookies
 - Clear storage → simulate logout
 - Manifest → PWA info

sessionStorage vs localStorage in auth

// Use sessionStorage for short-lived data

sessionStorage.setItem("temp_form", JSON.stringify(formData));

// Clears when tab closes – good for wizard forms

// Less persistent than localStorage

Note:

In DevTools → Application → localStorage → you can manually edit token → test privilege escalation!

5. Network Tab – Essential for Ethical Hacking & Debugging

Why it's important:

- See exact requests/responses
- Check headers (Authorization, Cookies, CORS)
- View response body (JSON, HTML)
- Check status codes (401 = unauth, 403 = forbidden)
- Measure timing (slow endpoints)
- Copy as cURL → replay in terminal/Burp

Common workflows

1. Login → watch Network tab → find auth request → copy token
2. Change role in localStorage → refresh → see if server trusts client
3. Look for API endpoints not linked in UI (hidden features)
4. Check for missing CORS headers
5. Spot large responses (data leakage)

Notes:

Filter by fetch/XHR → see only API calls

Click any request → Headers → Request Headers → see Authorization token

Summary:

- Full auth flow: Signup → Login → Store token → Use in headers → Logout
- APIs = how frontend talks to backend (REST + JSON most common)
- Authorization: JWT in header is standard – prefer HttpOnly cookies when possible
- DevTools Network tab = your #1 debugging + reconnaissance tool
- Storage: localStorage persistent, sessionStorage tab-only – both vulnerable to XSS